

ASSIGNMENT – 01

Q1. What are some popular HTTP libraries available across different programming languages (such as Python, Java, JavaScript, Ruby, PHP, C#, Go, Swift, Rust, and Perl), and how do these libraries act as implementations of the HTTP/1.1 protocol specification (e.g., RFC 7230)?

Across programming languages, HTTP libraries provide practical implementations of the HTTP/1.1 specification (defined in RFC 7230 and related RFCs). These libraries abstract low-level protocol details—like request formatting, headers, connections, and response parsing—into easy-to-use APIs.

Here's a language-wise overview with examples and how they map to HTTP/1.1:

◆ Python

requests – Most popular, user-friendly

http.client – Built-in low-level module

aiohttp – Async HTTP client/server

◆ Java

HttpURLConnection – Built-in

Apache HttpClient – Advanced features

OkHttp – Modern and efficient

◆ JavaScript (Node.js & Browser)

fetch API (browser & Node.js)

node:http – Core module

◆ Ruby

Net: HTTP – Built-in

HTTParty – Simple wrapper

◆ PHP

cURL – Widely used

Guzzle – Modern HTTP client

file_get_contents() – Basic requests

◆ **C#**

HttpClient – Standard modern library

WebRequest/WebClient – Older APIs

◆ **Go**

net/http – Standard library

◆ **Swift**

NSURLSession – Apple’s networking API

Alamofire – Popular wrapper

◆ **Rust**

reqwest – High-level client

hyper – Low-level, fast HTTP library

◆ **Perl**

LWP: User Agent – Popular HTTP client

HTTP: Tiny – Lightweight

How These Libraries implement HTTP/1.1(RFC 7230)?

Primary Functions:

- HTTP requests creation:
 - Generate HTTP requests: GET, POST, PUT, DELETE
 - Include headers, URL parameters, and message bodies
- Header Management:
 - Add, modify, and validate headers
 - Enforce rules like required Host header in HTTP/1.1
- Message Parsing:
 - Parse status line (HTTP/1.1 200 OK)

Extract headers and body

Handle chunked transfer encoding

- **Error Handling & Compliance:**

Follow protocol rules (e.g., malformed headers)

Handle redirects (3xx), client/server errors (4xx/5xx)

- **Transport Layer Integration:**

Use TCP sockets underneath

Support HTTPS via TLS

Example: (Python Requests Client)

The Requests library for Python is one of the best implementations.

Why Python Requests is Popular

- Easy and clean syntax
- Automatic handling of:

- o-Headers

- o-Cookies

- o-Converting JSON objects

- o-SSL certification

(Example Code)

```
import requests
```

```
response = requests.get("https://api.example.com/data")
```

```
print(response.status_code)
```

```
print(response.json())
```

Conclusion:

HTTP libraries are an indispensable tool that serves as a connector between developers and HTTP/1.1 protocol (RFC 7230). They provide a simple API for easy interaction with the web server on top of the implemented HTTP protocol. Libraries internally implement HTTP protocol.